

Лабораторна робота № 2

Мета роботи:

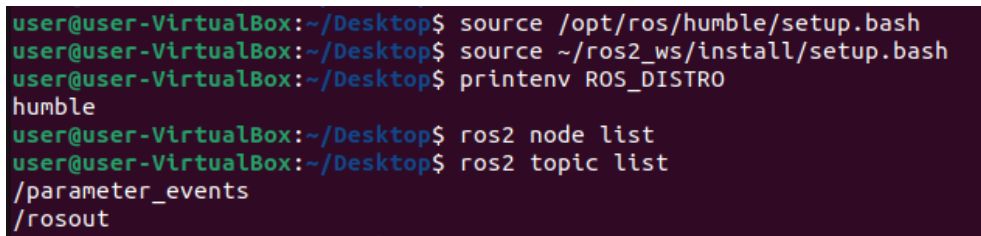
Сформувати практичні навички розробки ноди керування рухом у ROS2, навчитися працювати з повідомленням `geometry_msgs/Twist`, реалізувати підписку на `/cmd_vel`, обмеження швидкості та перевірити роботу ноди в симуляції.

Репозиторій з роботою: https://github.com/zakappa/ROS2_Ginzburg

Хід роботи

Підготовка середовища

Роботу виконано в тій самій віртуальній машині та робочому просторі, що й лабораторну роботу №1. ROS2-середовище активовано командами `source /opt/ros/humble/setup.bash` і `source ~/ros2_ws/install/setup.bash`. Змінна має значення `humble`. Команди `ros2 node list` і `ros2 topic list` виконуються без помилок. За відсутності запущених нод відображаються лише системні топіки (рис. 1).



```
user@user-VirtualBox:~/Desktop$ source /opt/ros/humble/setup.bash
user@user-VirtualBox:~/Desktop$ source ~/ros2_ws/install/setup.bash
user@user-VirtualBox:~/Desktop$ printenv ROS_DISTRO
humble
user@user-VirtualBox:~/Desktop$ ros2 node list
user@user-VirtualBox:~/Desktop$ ros2 topic list
/parameter_events
/rosout
```

Рисунок 1 - Активація середовища та перевірка команд ROS2

Для тестування запущено симулятор `turtlesim`. Після запуску в системі з'явилися нода `/turtlesim` і топік керування черепахою `/turtle1/cmd_vel` (рис. 2).

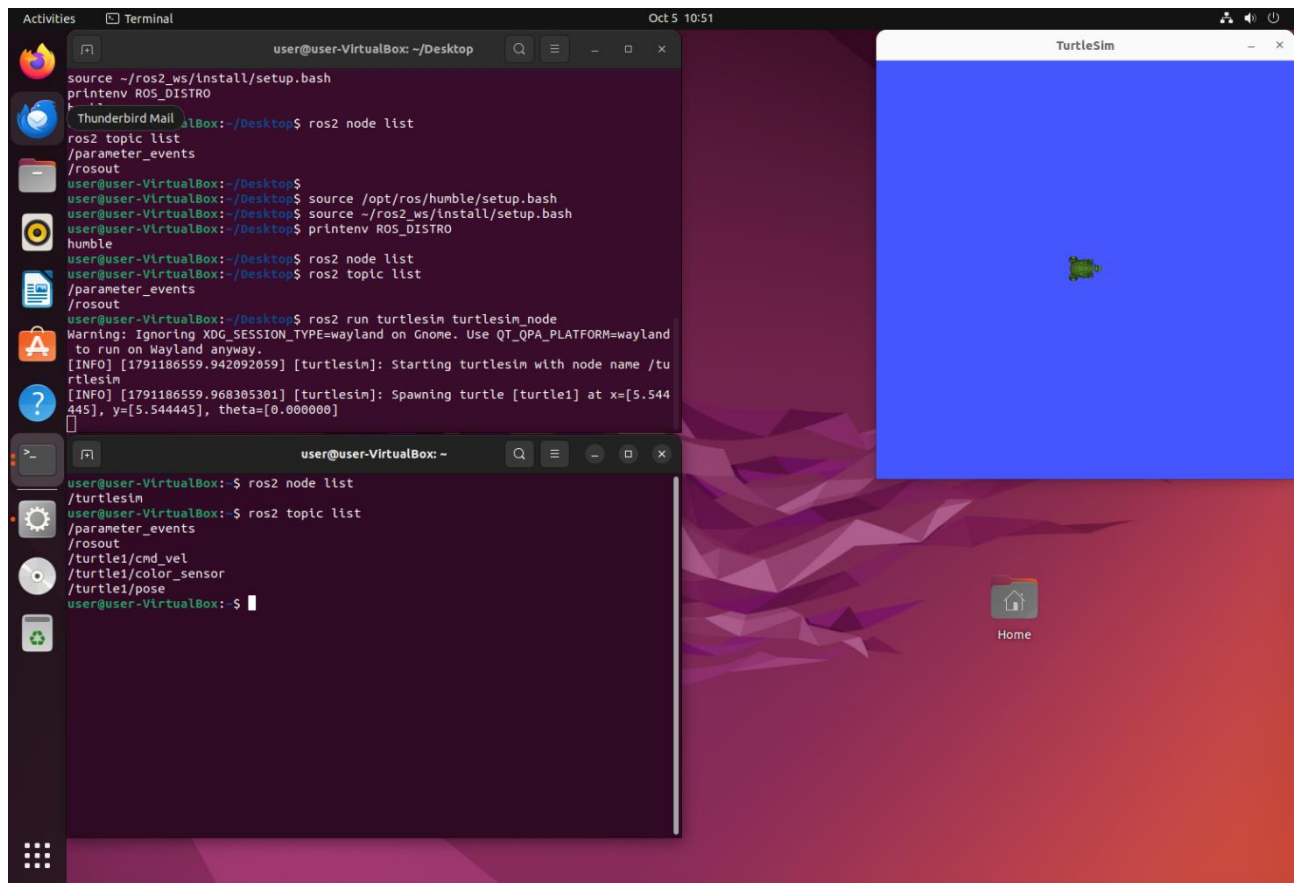


Рисунок 2 - Запуск симулятора turtlesim

Створення Motion Control

У каталозі `~/ros2_ws/src` створено пакет `motion_control` командою. Пакет має тип збірки `ament_python` і залежності `rcpp` та `geometry_msgs`. Параметр `--node-name` створив файл ноди `motion_node.py` і автоматично зареєстрував її в `setup.py` (рис. 3).

```
user@user-VirtualBox:~/ros2_ws/src$ tree ~/ros2_ws/src/motion_control
/home/user/ros2_ws/src/motion_control
├── LICENSE
├── motion_control
│   ├── __init__.py
│   └── motion_node.py
├── package.xml
├── resource
│   └── motion_control
├── setup.cfg
├── setup.py
├── test
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py
└── 3 directories, 10 files
```

Рисунок 3 - Створення пакету `motion_control` та його структура

Пакет зібрано командою `colcon build --packages-select motion_control`. Коректність встановлення перевірено командами `ros2 pkg list` і `ros2 pkg executables motion_control`, що показали наявність пакету та працюючого файлу `motion_node` (рис. 4).

```
user@user-VirtualBox:~/ros2_ws/src$ cd ~/ros2_ws
user@user-VirtualBox:~/ros2_ws$ colcon build --packages-select motion_control
Starting >>> motion_control
Finished <<< motion_control [0.97s]

Summary: 1 package finished [1.50s]
user@user-VirtualBox:~/ros2_ws$ source install/setup.bash
user@user-VirtualBox:~/ros2_ws$ ros2 pkg list | grep motion
motion_control
user@user-VirtualBox:~/ros2_ws$ ros2 pkg executables motion_control
motion_control motion_node
```

Рисунок 4 - Збірка робочого простору та перевірка наявності пакету

Реалізація Motion Node

Нода `motion_node` реалізована класом `MotionNode`, успадкованим від `rcslru.node.Node`, і виконує роль проміжного шару безпеки між джерелом команд і роботом.

Основні елементи реалізації:

- **Параметри:** задано пороги швидкостей (1.0 м/с і 1.5 рад/с) та топик призначення (`/turtle1/cmd_vel`).
- **Підписка:** прийом повідомлень `geometry_msgs/msg/Twist` із `/cmd_vel`.
- **Фільтрація:** метод `limit` обрізає перевищені `linear.x` та `angular.z` зі збереженням знака і видає `WARN`.
- **Публікація:** відправка безпечної команди у `/turtle1/cmd_vel`.
- **Логування:** фіксація вхідних і вихідних швидкостей на рівні `INFO`.

Фрагмент коду створення підписки на `/cmd_vel`:

```
from geometry_msgs.msg import Twist
```

```
class MotionNode(Node):
```

```
    def __init__(self):
```

```
        super().__init__('motion_node')
```

```
        self.declare_parameter('max_linear_speed', 1.0)
```

```
        self.declare_parameter('max_angular_speed', 1.5)
```

```
        self.declare_parameter('output_topic', '/turtle1/cmd_vel')
```

```

self.max_linear = self.get_parameter('max_linear_speed').value

self.max_angular = self.get_parameter('max_angular_speed').value

output_topic = self.get_parameter('output_topic').value


self.subscription = self.create_subscription(
    Twist, '/cmd_vel', self.cmd_vel_callback, 10)

self.publisher_ = self.create_publisher(Twist, output_topic, 10)

```

Фрагмент коду обробки повідомлення та обмеження швидкості:

```

def limit(self, name, value, max_value):

    if abs(value) <= max_value:

        return value

    # Обрізаємо до порогу, зберігаючи напрямок руху

    limited = math.copysign(max_value, value)

    self.get_logger().warn(
        f'{name}={value:.2f} exceeds limit +/-{max_value:.2f}, '
        f'limited to {limited:.2f}')

    return limited


def cmd_vel_callback(self, msg):

    self.get_logger().info(
        f'Received: linear.x={msg.linear.x:.2f}, angular.z={msg.angular.z:.2f}')


    cmd = Twist()

    cmd.linear.x = self.limit('linear.x', msg.linear.x, self.max_linear)

    cmd.angular.z = self.limit('angular.z', msg.angular.z, self.max_angular)

    self.publisher_.publish(cmd)


    self.get_logger().info(
        f'Sent: linear.x={cmd.linear.x:.2f}, angular.z={cmd.angular.z:.2f}')

```

Повний код ноди наведено на рис. 5 і в каталозі `code/motion_control` в репозиторії.

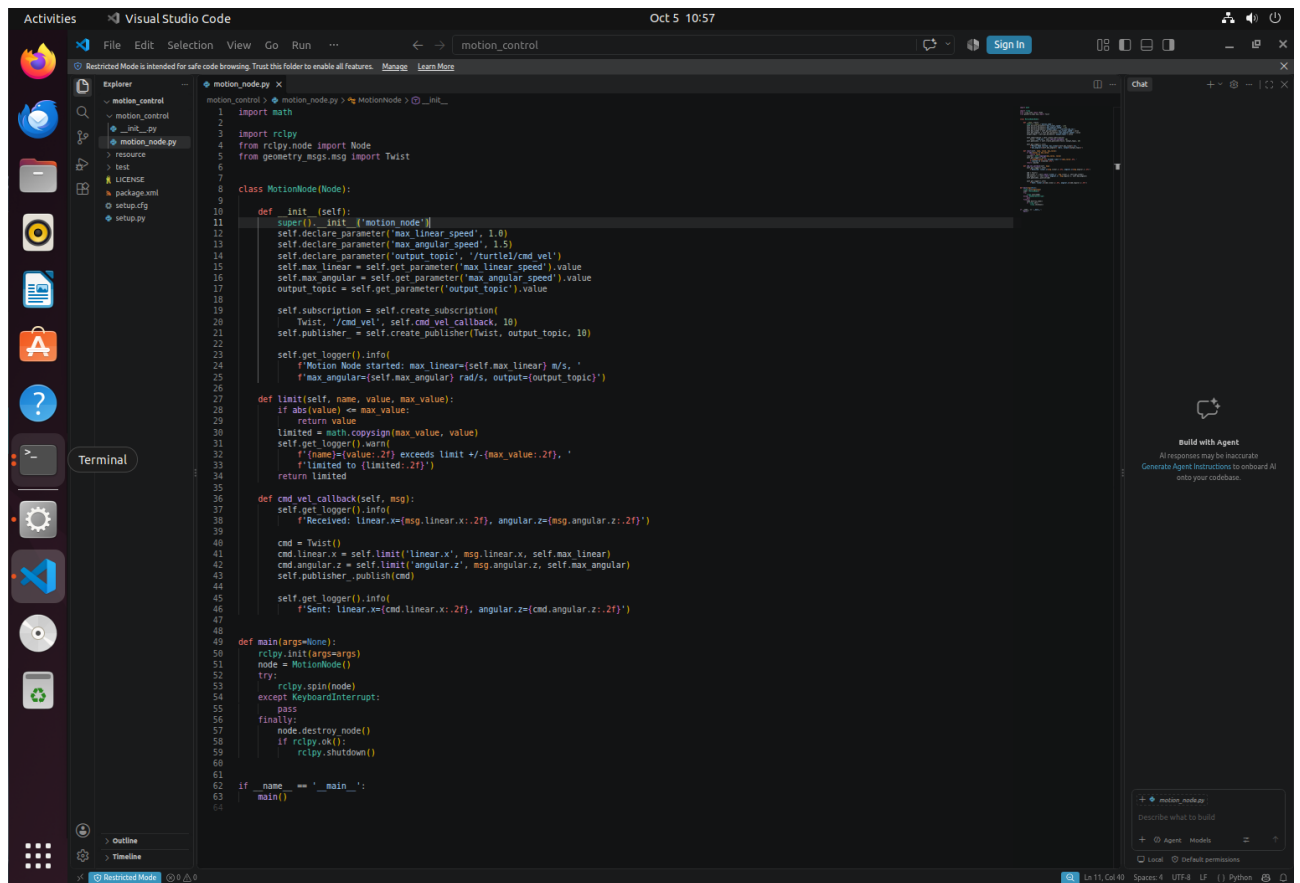


Рисунок 5 - Код motion_node

Залежність від geometry_msgs зафіксовано в package.xml, а точку входу зареєстровано в setup.py (рис. 6).

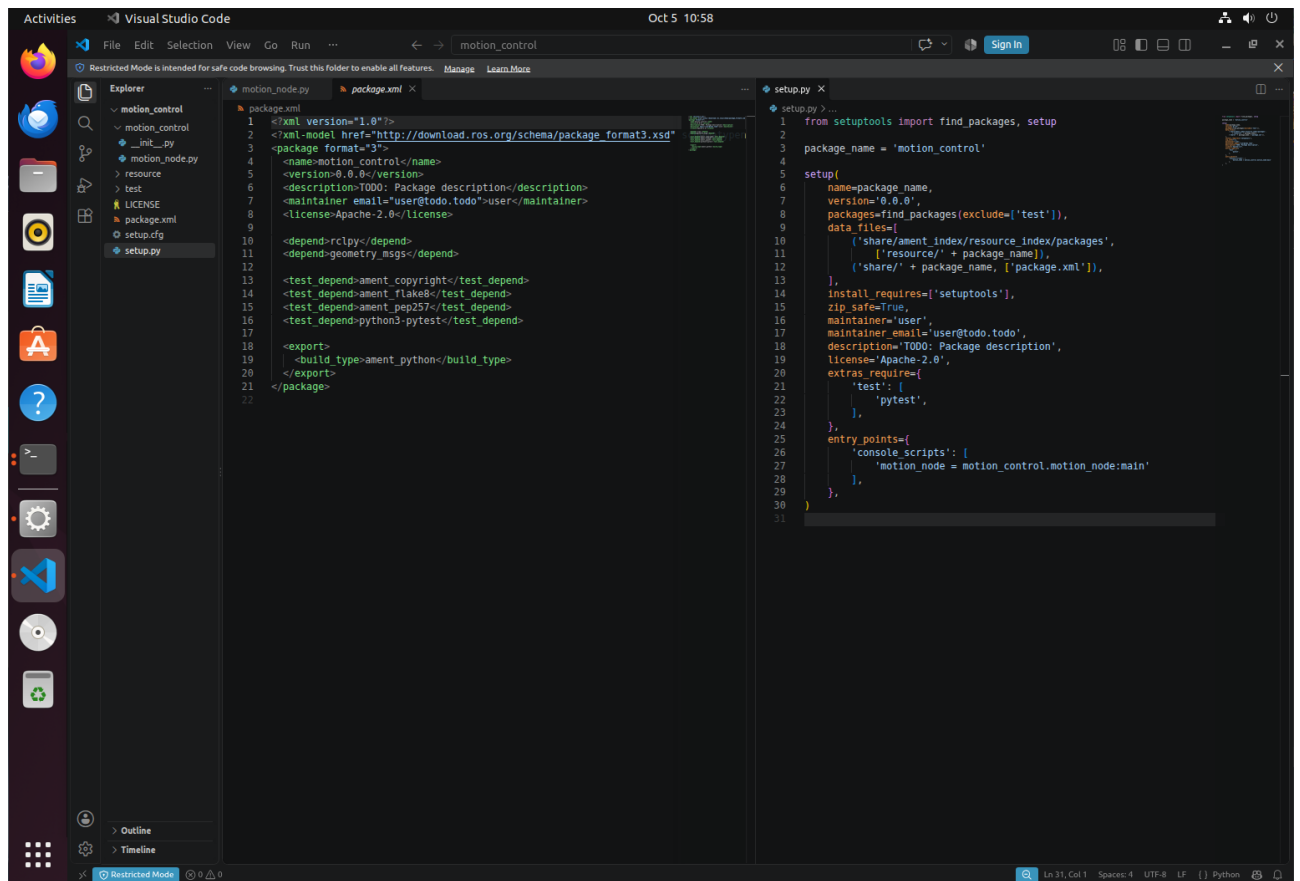


Рисунок 6 - Залежності package.xml та реєстрація ноди

Тестування ноди

Під час старту нода виводить у журнал встановлені пороги та вихідний топик. Команда `ros2 node info /motion_node` підтвердила, що нода підписана на `/cmd_vel` і публікує в `/turtle1/cmd_vel` (рис. 7).

```
user@user-VirtualBox:~$ ros2 node list
/motion_node
/turtlesim
user@user-VirtualBox:~$ ros2 node info /motion_node
/motion_node
Subscribers:
  /cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Service Servers:
  /motion_node/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /motion_node/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /motion_node/get_parameters: rcl_interfaces/srv/GetParameters
  /motion_node/list_parameters: rcl_interfaces/srv/ListParameters
  /motion_node/set_parameters: rcl_interfaces/srv/SetParameters
  /motion_node/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

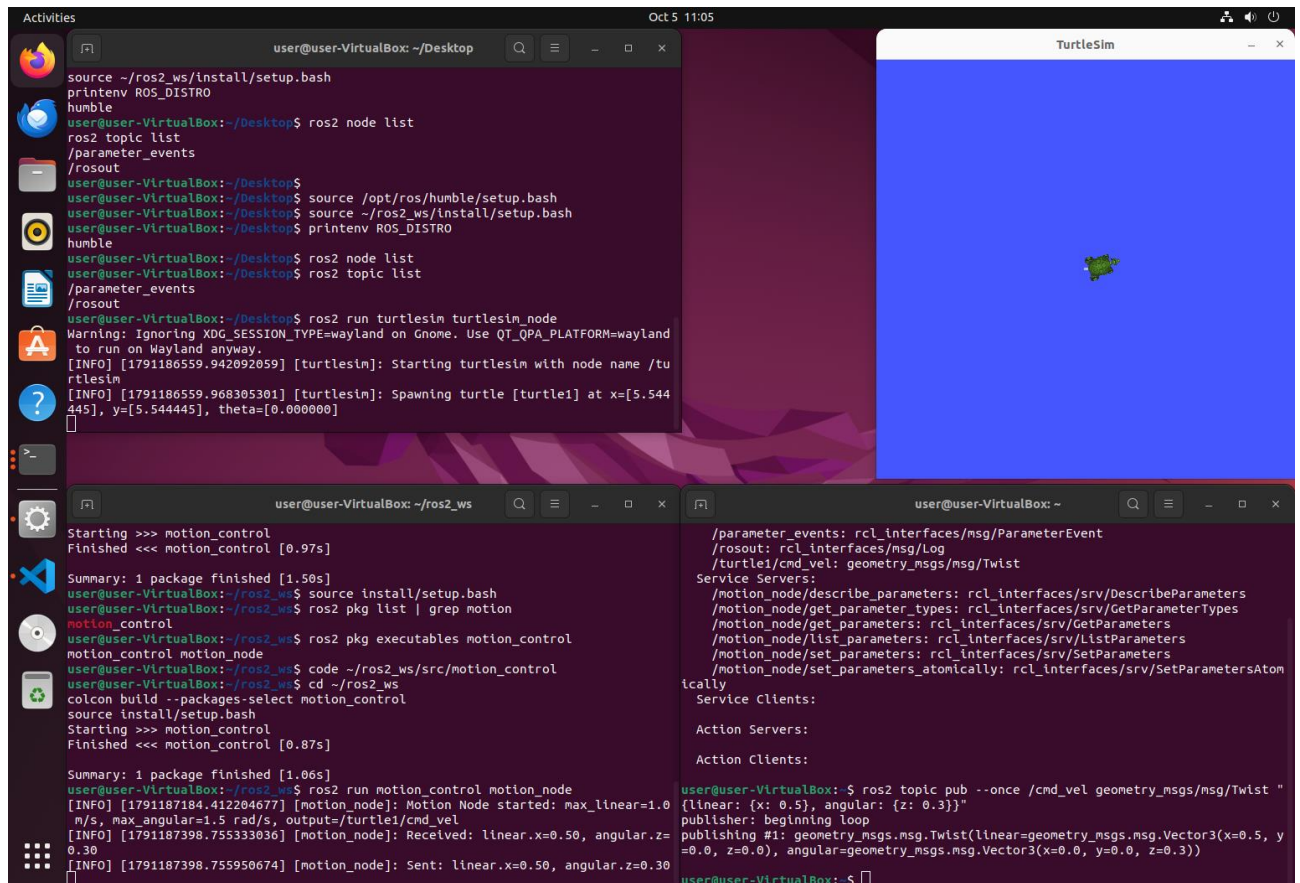
Action Servers:

Action Clients:
```

Рисунок 7 - Запуск motion_node та інформація про неї

Команди в `/cmd_vel` подавалися двома способами: програмно через `ros2 topic pub` і вручну через `turtle_teleop_key`, перенаправлений на топик `/cmd_vel` (`--ros-args -r /turtle1/cmd_vel:=/cmd_vel`).

Під час подачі команди в межах допустимих значень (`linear.x = 0.5`, `angular.z = 0.3`) нода вивела отримані значення на рівні INFO і передала їх у без змін (рис. 8).



```
user@user-VirtualBox: ~/Desktop
source ~/ros2_ws/install/setup.bash
printenv ROS_DISTRO
humble
user@user-VirtualBox:~/Desktop$ ros2 node list
ros2 topic list
/parameter_events
/rostop
user@user-VirtualBox:~/Desktop$ source /opt/ros/humble/setup.bash
user@user-VirtualBox:~/Desktop$ source ~/ros2_ws/install/setup.bash
user@user-VirtualBox:~/Desktop$ printenv ROS_DISTRO
humble
user@user-VirtualBox:~/Desktop$ ros2 node list
user@user-VirtualBox:~/Desktop$ ros2 topic list
/parameter_events
/rostop
user@user-VirtualBox:~/Desktop$ ros2 run turtlesim turtlesim_node
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland
to run on Wayland anyway.
[INFO] [1791186559.942092059] [turtlesim]: Starting turtlesim with node name /tu
rtlesim
[INFO] [1791186559.968305301] [turtlesim]: Spawning turtle [turtle1] at x=[5.544
445], y=[5.544445], theta=[0.000000]
user@user-VirtualBox:~/Desktop$

Starting >>> motion_control
Finished <<< motion_control [0.97s]
Summary: 1 package finished [1.50s]
user@user-VirtualBox:~/ros2_ws$ source install/setup.bash
user@user-VirtualBox:~/ros2_ws$ ros2 pkg list | grep motion
motion_control
user@user-VirtualBox:~/ros2_ws$ ros2 pkg executables motion_control
motion_control motion_node
user@user-VirtualBox:~/ros2_ws$ code ~/ros2_ws/src/motion_control
user@user-VirtualBox:~/ros2_ws$ cd ~/ros2_ws
colcon build --packages-select motion_control
source install/setup.bash
Starting >>> motion_control
Finished <<< motion_control [0.87s]
Summary: 1 package finished [1.06s]
user@user-VirtualBox:~/ros2_ws$ ros2 run motion_control motion_node
[INFO] [1791187184.412204677] [motion_node]: Motion Node started: max_linear=1.0
m/s, max_angular=1.5 rad/s, output=/turtle1/cmd_vel
[INFO] [1791187398.755333036] [motion_node]: Received: linear.x=0.50, angular.z=
0.30
[INFO] [1791187398.755950674] [motion_node]: Sent: linear.x=0.50, angular.z=0.30
user@user-VirtualBox:~/ros2_ws$

/parameter_events: rcl_interfaces/msg/ParameterEvent
/rostop: rcl_interfaces/msg/Log
/turtle1/cmd_vel: geometry_msgs/msg/Twist
Service Servers:
/motion_node/describe_parameters: rcl_interfaces/srv/DescribeParameters
/motion_node/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
/motion_node/get_parameters: rcl_interfaces/srv/GetParameters
/motion_node/list_parameters: rcl_interfaces/srv/ListParameters
/motion_node/set_parameters: rcl_interfaces/srv/SetParameters
/motion_node/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:
Action Servers:
Action Clients:
user@user-VirtualBox:~/ros2_ws$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "
{linear: {x: 0.5}, angular: {z: 0.3}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y
=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.3))
user@user-VirtualBox:~/ros2_ws$
```

Рисунок 8 - Вивід логів ноди при команді в межах допустимих значень

Для перевірки обмеження швидкості вказано команди з перевищенням обмежень. В обох випадках нода обмежила значення до зазначеного порогу зі збереженням руху й вивела WARN-повідомлення (рис. 9).

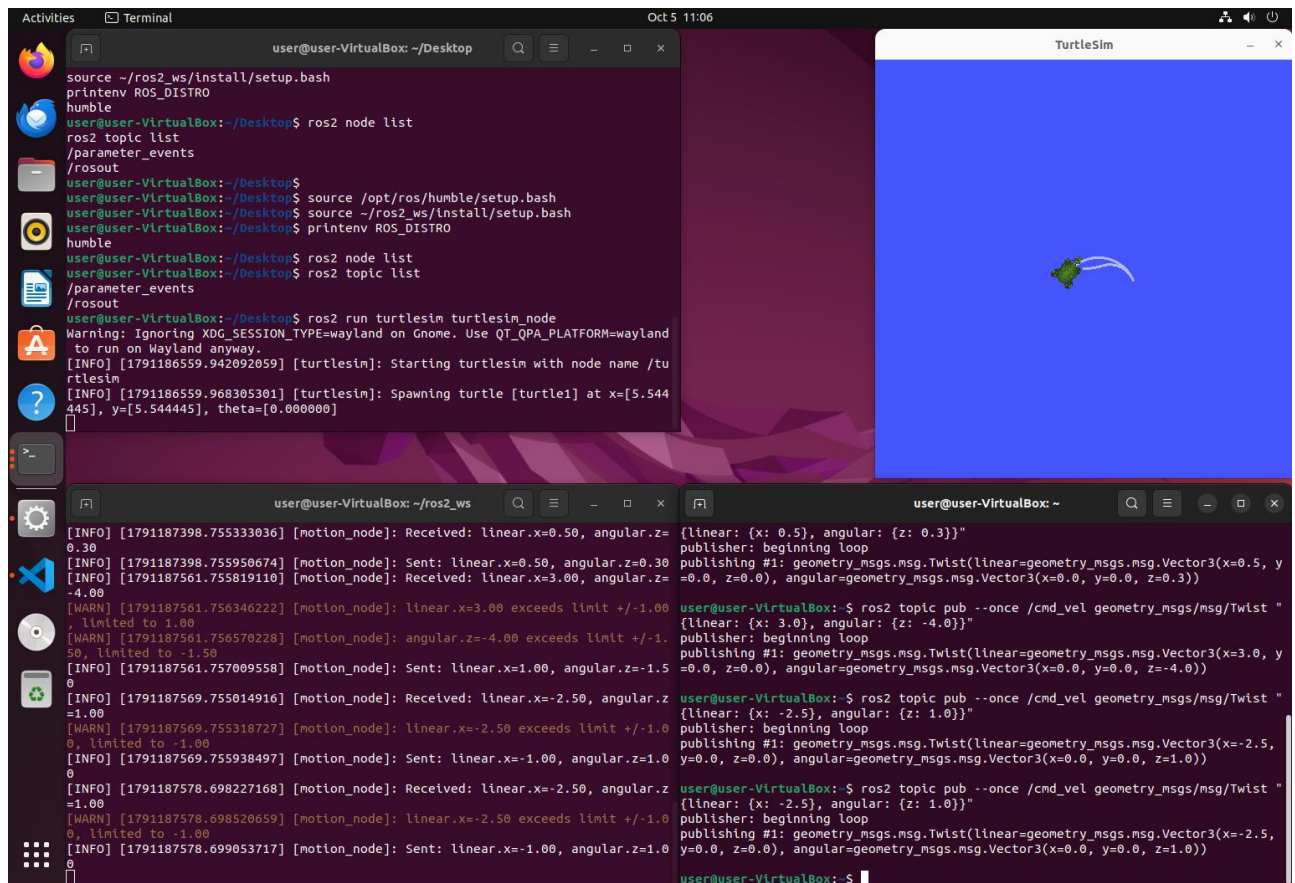


Рисунок 9 - Демонстрація обмеження швидкості

Для перевірки через ros2 топіс echo одночасно переглядалися вхідний топік ноди і вихідний. При поданні команди $\text{linear.x} = 3.0$, $\text{angular.z} = -4.0$ у вхідному топіку видно вихідні значення, а у вихідному обмежені 1.0 і -1.5 (рис. 10).

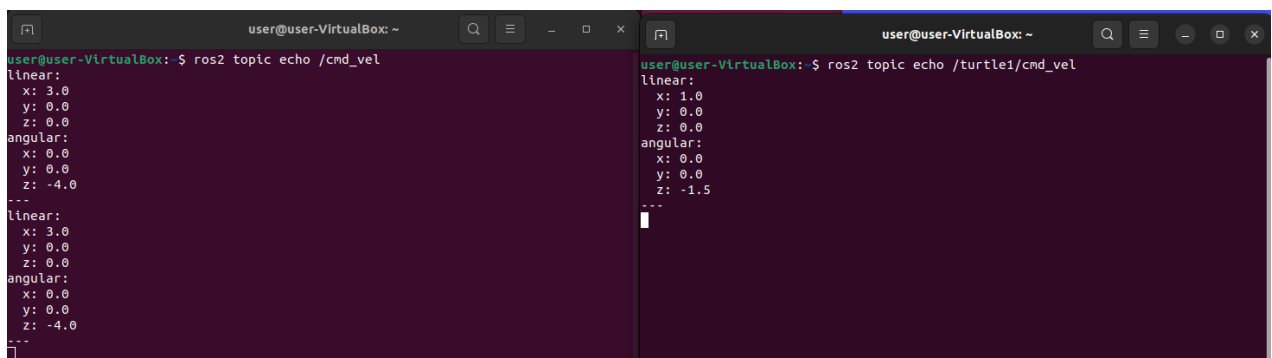


Рисунок 10 - Перевірка через ros2 топіс echo: вхідні та обмежені команди

Під час ручного керування через teleop кожне натискання клавіші надсилає швидкість 2.0, що перевищує ліміти. Нода обмежувала її до 1.0

(лінійна) та 1.5 (кутова), тому черепаха рухалася помітно повільніше, ніж при прямому керуванні в лабораторній роботі №1 (рис. 11).

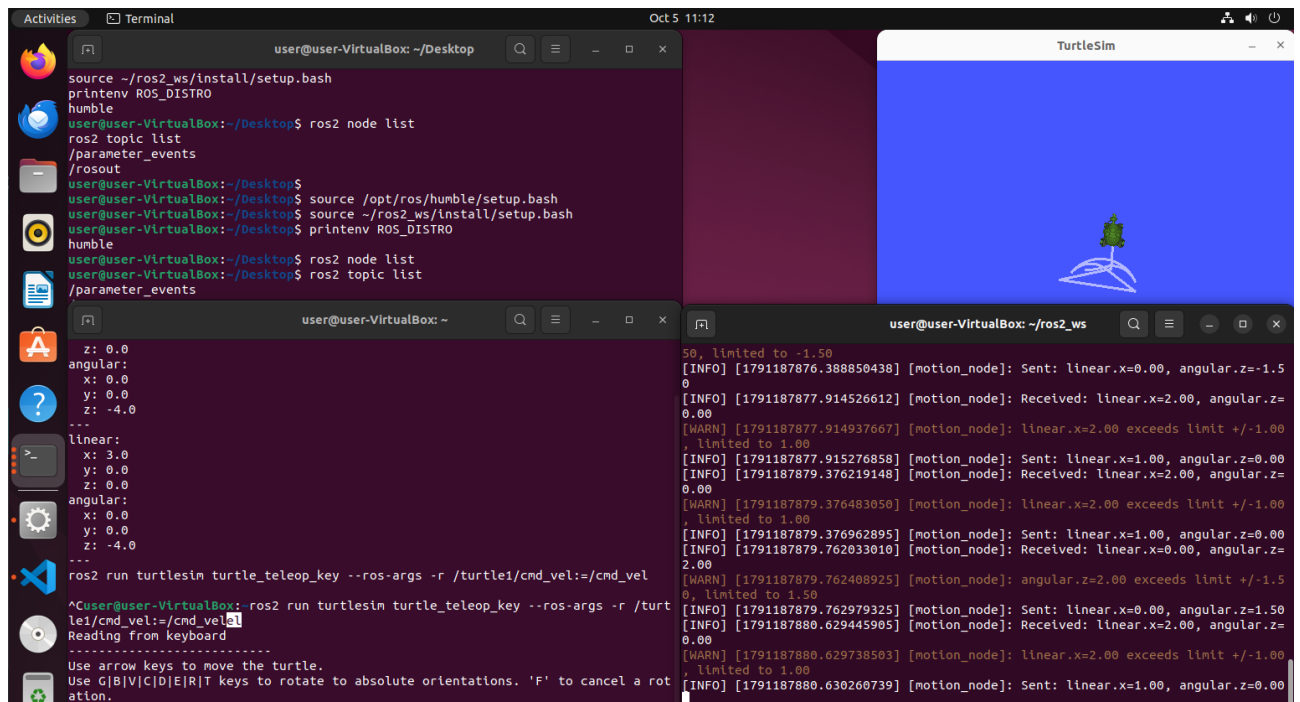


Рисунок 11 - Ручне керування через teleop з лімітом швидкості

Висновок

У ході лабораторної роботи розроблено ноду керування рухом у пакеті motion_control. Нода підписується на стандартний топік /cmd_vel, обробляє поля linear.x і angular.z повідомлення geometry_msgs/msg/Twist, виводить їх у журнал на рівні INFO, обмежує швидкості до допустимих лімітів із виводом WARN-повідомлень і передає безпечну команду симулятору. Роботу ноди перевірено в turtlesim командами ros2 topic pub, ros2 topic echo, ros2 node info.

Порівняння ручного та програмного керування:

- **Ручне керування (teleop):** команди задає оператор у реальному часі. Забезпечує високу гнучкість у непередбачених ситуаціях, але має низьку точність і фіксовані кроки швидкості.
- **Програмне керування (ros2 topic pub):** команди генеруються кодом автономно. Забезпечує високу точність і повторюваність траєкторії, проте критично залежить від коректності алгоритму.
- **Роль motion_node:** виступає єдиним шлюзом безпеки незалежно від джерела команд, нода фільтрує небезпечні значення швидкостей і логує відхилення в одному місці.